AXON-X

AUTONOMOUS

Master plan for bounded DashPro autonomy and the Axon-X mobile control plane.

Generated 2026-07-21 11:48

Source: docs/AXON-X-AUTONOMY-MASTER-PLAN.md

Assessment: docs/AXON-X-AUTONOMY-READINESS.md

Axon-X Autonomy Master Plan

Purpose: Turn the readiness assessment into a strict, ordered build plan.
Source of truth: `docs/AXON-X-AUTONOMY-READINESS.md`
Created: 20 July 2026
Target: Bounded autonomy so any bound workspace can work in its own project/app, proven first on DashPro, then a secure Axon-X mobile control plane

---

Mission

Build a system where:

1. Workspaces can work on/in their own projects and apps without a human babysitting every step.
2. Axon-X can build and operate its own mobile control plane under the same safety rules.
3. Dangerous actions stay blocked unless a human explicitly approves them.

This is **bounded autonomy**, not unlimited power. DashPro is the first proving ground for the multi-workspace rule; the same controls must apply to every bound workspace (including `workspace_axon_watch`).

North-star outcome

```
Approved goal
  → Lead divides work
  → One leased task
  → Isolated branch / worktree
  → Specialist implements
  → Mandatory tests pass
  → Independent AI review
  → Draft pull request
  → CI watch + limited repair
  → Staging deploy + health check
  → Human gate for production risk
  → Release or automatic rollback
  → Receipt for every step
```

Until that loop is proven, continuous workers must not freely edit any live bound workspace checkout (DashPro is the highest-risk example today).

Non-negotiable rules

RULE	WHY
Order is mandatory	Later stages depend on earlier safety
No live shared checkout for continuous workers	Prevents mixed, unowned changes
No anonymous mutation APIs	Remote and mobile control must be safe
Tests are chosen before work starts	Workers cannot grade their own homework
Pull requests before merge	Dirty folders are not a delivery method
Production stays human-gated first	Secrets, store promotion, and irreversible actions remain protected
Pause before expand	Do not add autonomy while dirty live canary trees are unsorted
Post-receipt Critical Review Clause	Every agent (incl. Verifier when Gate 6 lands) ends with rewrite + `Confidence: N/10`; missing confidence fails closed
Workspace-scoped platform capability	DashPro is Wave A canary; the same loop applies to every bound `workspace_id`

Current starting point (from readiness)

METRIC	VALUE
Overall safe production autonomy	~35–38%
Supervised operator maturity	~68%
Safe task → PR loop	~25–38%
Secure mobile control plane	~10%
Immediate hazard	DashPro live checkout has a large mixed change set
Immediate blocker	Usage limits + shared capacity + restart cancellations

Timeline overview

These calendar ranges are **rough planning guesses**, not commitments. They assume parallel engineering, available Cursor capacity, and no major scope expansion.

PHASE	GATES	FOCUS	ROUGH CALENDAR GUESS
Stabilize	0–1	Stop damage, restore trust	~Week 1
Contain	2–3	Auth + isolation	~Weeks 2–4
Orchestrate	4–7	Tasks, Lead, verify, review	~Weeks 4–8
Publish	8–10	PRs, CI loop, fair scheduling	~Weeks 8–12
Prove	11–12	Staging + DashPro canary	~Weeks 12–16
Remote	13–14	Mobile PWA + bounded production	~Weeks 16–20

Parallel work is allowed only when earlier gates are green and the debt rules below are met.

Debt gate (must stay green)

Do not start the next major gate while any of these are true:

- continuous workers are mutating a shared live checkout;
- autonomy-critical tests are red;
- review-ready backlog is uncontrolled;
- zombie `executing` runs exist;
- Cursor usage is exhausted with no recovery policy;
- anonymous network clients can mutate the control plane.

Gate-by-gate plan

Gate 0 — Pause and preserve

Priority: P0
Estimate: 1–2 days
Depends on: nothing

Goal

Stop unattended mutation and protect every current change.

Work

1. Disable continuous worker mutation for DashPro (and Axon-X if needed) via scheduler / employee toggles.
2. Snapshot run receipts for every recent DashPro and Axon-X employee run.
3. Inventory every changed path in `/home/edp/Projectx/product/dashpro`.
4. Map each path to a run, employee role, or “unknown”.
5. Decide keep / discard / hold for each cluster of changes.
6. Preserve both dirty trees; do not reset or force-clean without operator confirmation.

Exit criteria

- Scheduler mutation is paused where required.
- Every changed DashPro path has an owner and disposition.
- No worker is writing while triage is active.

Evidence to record

- scheduler status JSON;
- change inventory markdown;
- mapping of paths → run IDs / roles.

Gate 1 — Restore a trustworthy baseline

Priority: P0
Estimate: 2–4 days
Depends on: Gate 0

Goal

One known-good commit where autonomy foundations are green.

Work

1. Re-run autonomy-critical modules on a clean comparison or isolated DB fixture set and record pass/fail with SHA.
2. Fix any remaining scheduler-settings test isolation, roster active-run projection, retention pruning, and missing imports if they reproduce.
3. Run supported backend contract suite.
4. Run console typecheck, Vitest, and production build.
5. Run contracts / preflight gates without raising budgets.
6. Record HEAD SHA, command outputs, and pass counts.

Exit criteria

- Backend contract runner green at one recorded commit.
- Console typecheck / tests / build green at that same commit.
- Autonomy-critical modules green when run the supported way (not only one ad-hoc invocation).

Evidence to record

- `verify:contracts` result;
- Vitest summary;
- typecheck / build logs;

- commit SHA.

Gate 2 — Authentication and containment

Priority: P0

Estimate: 7–12 days

Depends on: Gate 1

Goal

Make powerful actions attributable and least-privilege.

Work

1. Authenticate every mutating control-plane API (runs, chat dispatch, terminals, fleet, vault proxy, tunnel).
2. Keep watch service internal-only with service identity / mTLS or equivalent.
3. Disable vault auto-unlock in any remotely reachable deployment.
4. Add step-up approval for Full Access and exact-effect actions.
5. Restrict scheduled Cursor flags (`--trust`, `--force`, `--approve-mcps`) to a narrower worker policy.
6. Add rate limits, CSRF / origin protection, and audit identity fields.

Exit criteria

- Anonymous mutation is impossible.
- Every remote action has a revocable identity.
- Workers run least-privilege by default.
- Vault unlock is explicit and audited.

Evidence to record

- auth middleware tests;
- denied-anonymous mutation proofs;
- vault unlock audit receipts;
- worker trust-policy tests.

Gate 3 — Per-task disposable checkout

Priority: P0

Estimate: 4–7 days

Depends on: Gates 1 and 2

Goal

Every continuous worker gets its own branch and worktree.

Work

1. Reuse safe-improvement isolation patterns for normal worker runs.
2. Create one branch + git worktree per leased task / run.
3. Pin baseline SHA before edits.
4. Forbid writes outside the disposable root.
5. Clean up failed worktrees with receipts.
6. Keep the operator live checkout read-only for continuous workers.

Exit criteria

- Two workers can change the same repository without sharing a working tree.
- Operator checkout remains untouched by continuous workers.
- Failed tasks leave cleanup receipts, not dirty shared folders.

Evidence to record

- isolation unit tests;
- concurrent-worker proof;
- before/after `git status` on the live root.

Gate 4 — Durable task ledger

Priority: P0

Estimate: 4–7 days

Depends on: Gate 3

Goal

Replace “pick the highest-value task” with leased, recorded work.

Work

1. Add task model: goal, acceptance criteria, risk, owner role, dependencies, lease, attempt budget, terminal outcome.
2. Create APIs / store for task create, lease, complete, fail, cancel.
3. Require every worker run to reference exactly one leased task.
4. Block untracked self-selected continuous work.
5. Surface task board in Mission Control / company roster.

Exit criteria

- Every worker run is created from one leased task.
- No continuous shift starts without a task ID.
- Attempt budgets and leases are enforced.

Evidence to record

- task schema + migration;
- lease contention tests;
- UI screenshot or API snapshot of a leased task.

Gate 5 — Lead planner and conflict policy

Priority: P0
Estimate: 5–8 days
Depends on: Gate 4

Goal

Make Dana (Lead) a real manager, not only a persona.

Work

1. Convert approved goals / backlog items into a dependency DAG of tasks.
2. Assign roles (frontend / backend / integrations / watcher).
3. Serialize overlapping file paths.
4. Cancel obsolete tasks when goals change.
5. Persist replan receipts.

Exit criteria

- One goal produces an ordered task plan.
- Overlapping edits cannot run concurrently.
- Replans are receipt-backed.

Evidence to record

- planner unit tests;
- conflict-serialization proof;
- sample goal → task DAG artifact.

Gate 6 — Mandatory verifier contract

Priority: P0
Estimate: 5–8 days
Depends on: Gate 4

Goal

Workers cannot finish without machine-checkable proof.

Work

1. Generate task-specific checks before execution.
2. Require lint / type / test / build / security / diff-budget checks as applicable.
3. Keep the verifier immutable to the worker.
4. Block `review_ready` / PR creation when checks fail.
5. Persist command outputs as receipts.

Exit criteria

- No task reaches review-ready without acceptance evidence.
- Failed checks always block publishing.
- Diff policy rejects secrets and out-of-scope paths.

Evidence to record

- verifier contract tests;

- fail-closed examples;
- sample acceptance receipt.

Gate 7 — Independent review agent

Priority: P1

Estimate: 3–5 days

Depends on: Gate 6

Goal

A second agent reviews the change; the editor is not the only judge.

Work

1. Run defect-first review in a separate context / role.
2. Block on actionable findings.
3. Cap repair ↔ review cycles.
4. Attach reviewer findings to the task and PR.

Exit criteria

- Every candidate has reviewer findings and a final pass / fail.
- Repair loops stop after the attempt budget.

Evidence to record

- review agent tests;
- sample finding → repair → pass trail.

Gate 8 — Automated branch and PR lifecycle

Priority: P1

Estimate: 4–7 days

Depends on: Gates 3, 6, 7

Goal

Successful tasks end as clean draft PRs.

Work

1. Create scoped commits on the task branch.
2. Push the branch.
3. Open a draft PR with evidence links.
4. Update the same PR from review / CI repairs.
5. Never merge protected branches autonomously.
6. Clean up failed worktrees and abandon branches with receipts.

Exit criteria

- Successful tasks leave a draft PR, not a dirty live tree.
- Failed tasks leave a cleaned-up worktree.
- Protected merges remain human-gated.

Evidence to record

- PR-create integration tests;
- sample draft PR URL;
- cleanup receipts.

Gate 9 — Closed CI remediation loop

Priority: P1
Estimate: 5–8 days
Depends on: Gates 4 and 8

Goal

CI red becomes a durable repair loop, not a chat prompt.

Work

1. Ingest GitHub check / workflow events for DashPro.
2. Classify the first hard-failing step.
3. Lease a repair task to the correct role.
4. Rerun the exact failing check.
5. Deduplicate alerts and escalate after attempt budget.

Exit criteria

- A deliberately broken test is detected, repaired on the task branch, rerun, and reflected on the PR without a new human prompt.

Evidence to record

- webhook handler tests;
- broken-test drill log;
- attempt-budget escalation proof.

Gate 10 — Durable execution and fair scheduling

Priority: P1
Estimate: 5–8 days
Depends on: Gate 4

Goal

Restarts and capacity limits no longer destroy progress or starve DashPro.

Work

1. Persist task leases and checkpoints.
2. Add per-workspace quotas and weighted fairness.
3. Resume safe tasks after control-plane restart.
4. Enforce token / cost / time budgets.
5. Keep usage-limit suppression, but with recovery and operator notice.

Exit criteria

- Restart drills recover without duplicate work.
- One workspace cannot occupy all worker slots.
- Usage exhaustion is visible and recoverable.

Evidence to record

- restart recovery drill;
- fairness simulation / test;

- budget enforcement tests.

Gate 11 — Staging deploy and rollback

Priority: P1

Estimate: 7–12 days

Depends on: Gates 2, 8, 9, 10

Goal

Prove the running app, not only the diff.

Work

1. Provision isolated staging for DashPro (and later Axon-X mobile).
2. Deploy immutable artifacts.
3. Run smoke / health checks.
4. Canary a small slice of traffic or synthetic checks.
5. Auto-rollback on failure.
6. Keep production promotion approval-gated.

Exit criteria

- A bad candidate rolls back automatically.
- Production remains approval-gated.
- Deployment receipts are retained.

Evidence to record

- staging deploy runbook;
- rollback drill;
- health-check receipts.

Gate 12 — DashPro autonomy canary

Priority: P1

Estimate: 2–3 weeks elapsed

Depends on: Gates 5–11

Goal

Prove the architecture with 20 real but bounded tasks.

Task mix

- UI bug;
- API bug;
- CI repair;
- dependency / config fix;
- ambiguous / failing cases;
- no-op / already-fixed cases.

Success bar

- ≥ 90% task completion;
- 0 live-tree corruption;
- 0 unauthorized effects;
- bounded cost;
- no unresolved duplicate execution;
- human interventions counted and declining.

Evidence to record

- canary scorecard;
- cost / intervention chart;
- incident list (if any).

Gate 13 — Mobile web control plane (PWA first)

Priority: P2
Estimate: 2–4 weeks
Depends on: Gates 2, 10, 12

Goal

A registered phone can supervise and approve exact actions safely.

Work

1. Package installable PWA from the existing `/mobile` shell.
2. Add authenticated push deep-links.
3. Ship run / approval / inbox views.
4. Add reconnect semantics and device revocation.
5. Do **not** claim background listening.
6. Defer native app until PWA proves the control model.

First mobile release must support

1. read fleet health;
2. read active-run evidence;
3. receive high-priority alerts;
4. stop a run;
5. approve or reject one exact action;
6. revoke a lost device.

Exit criteria

- A registered phone can inspect evidence, stop a run, and approve an exact effect.
- Anonymous phones cannot mutate anything.

Evidence to record

- PWA install proof;
- device enrollment / revocation tests;
- mobile approval drill.

Gate 14 — Bounded production autonomy

Priority: P2
Estimate: 2–4 weeks elapsed
Depends on: Gates 11–13

Goal

Allow only reversible, pre-approved production classes.

Allowed later (examples)

- restart a known healthy service;
- redeploy a previously approved artifact to staging;
- open a draft PR;
- rerun a failed CI job.

Forever human-gated

- production secrets;
- approval-policy changes;
- destructive data operations;
- force-push;
- protected-branch merge;
- Play / App Store promotion;
- irreversible migrations;
- expansion of the system’s own authority.

Exit criteria

- Production canary passes rollback drills.
- Authority can be globally revoked in one action.
- Incident review board signs the policy.

Workstream map

WORKSTREAM	OWNS GATES	PRIMARY OWNERS (ROLES)
Stabilize & evidence	0–1	Lead + Integrations
Security containment	2	Backend + Integrations
Isolation fabric	3	Backend
Tasking & planning	4–5	Lead + Backend
Quality & review	6–7	Backend + Watcher
Publish & CI	8–9	Integrations + Backend
Reliability	10	Backend + Watcher
Release proof	11–12	Lead + Integrations
Mobile remote	13–14	Frontend + Integrations + Lead

Definition of done for “workspace autonomy”

A **bound workspace** (any project-path workspace, not only DashPro) is autonomous enough only when all of the following are true for that workspace:

1. Continuous workers never write the live operator checkout.
2. Every run is tied to a leased task with acceptance checks.
3. Lead (or equivalent planner) can turn one approved goal into a conflict-safe task plan.
4. Draft PRs are the normal delivery unit.
5. CI failures create repair tasks automatically, with attempt budgets.
6. Staging rollback works without a human present.
7. A bounded canary (for example 20 tasks) meets the success bar for that workspace.
8. Merge to protected branches, store promotion, secrets, and destructive ops remain human-gated.

DashPro proving-ground note

DashPro is the first workspace used to prove the loop above. Passing the DashPro canary unlocks the pattern; it does **not** automatically declare every other bound workspace done until the same controls are enabled and measured there.

Definition of done for “Axon-X mobile control plane”

Mobile autonomy / control is **done** only when:

1. Auth and device enrollment are mandatory.
2. PWA is installable and usable on a phone.
3. Push alerts deep-link to the exact run or approval.
4. A phone can stop work and approve exact effects.
5. A lost phone can be revoked immediately.
6. No background-listening claim is made.
7. Native packaging is an optional later product decision, not a blocker.

Weekly operating cadence

CADENCE	ACTION
Daily	Check debt gate, scheduler status, usage limits, failed runs
End of each gate	Write exit evidence into this plan’s evidence log
Weekly	Operator review: keep / slip / cut scope
After Gate 12	Freeze architecture; only harden and measure
After Gate 14	Authority review and rollback drill

Evidence log template

Append one block per completed gate:

```
### Gate N — YYYY-MM-DD
- owner:
- commit:
- commands run:
- pass/fail:
- exit criteria met: yes/no
- residual risks:
- next gate unlocked:
```

Immediate next actions (start now)

1. **Pause** continuous mutation affecting the shared DashPro checkout.
2. **Inventory** the current DashPro dirty tree and map files to runs/roles.
3. **Fix** the failing autonomy tests and record a green baseline SHA.
4. **Do not** build mobile mutation features until Gate 2 is green.
5. **Do not** re-enable continuous live-checkout editing after Gate 0.

Related documents

- Assessment: `docs/AXON-X-AUTONOMY-READINESS.md`
- Interactive canvas: workspace canvases `axon-x-autonomy-readiness.canvas.tsx`
- Self-improvement contract: `docs/SELF_IMPROVEMENT_CONTRACT.md`
- DashPro CI playbook: `docs/planning/DASHPRO_CI_AGENT_PLAYBOOK.md`
- Child project binding: `docs/CHILD_PROJECT_WORKSPACE.md`

Appendix — Autonomy readiness assessment

Axon-X Autonomy Readiness

Plain-language assessment focused on DashPro as the first proving ground, plus the Axon-X mobile control plane. The autonomy target is multi-workspace: any bound workspace working in its own project/app under the same safety rules.
Assessment date: 20 July 2026 (live evidence below is a snapshot; re-check before acting)
Repository: `axon-watch`
Current branch during assessment: `dev`

The short answer

Axon-X is a real working system, not just a visual demonstration.

It can already:

- connect to the real DashPro project;
- launch AI workers that can read and change files;
- run several specialist workers on a schedule;
- remember whether work is running, finished, stopped, or failed;
- show activity and warnings in a control panel;
- run tests and record evidence;
- monitor parts of DashPro such as Sentry, PostHog, and Supabase;
- provide a basic phone-sized browser screen.

But it is **not yet safe to leave completely alone**.

These percentages are **judgment scores from a 20 July 2026 audit**, not measured completion of a formal rubric. Different reviewers used slightly different lenses (about **38%** for closed-loop DashPro autonomy vs about **68%** for “supervised operator platform”). Treat them as order-of-magnitude guidance only:

GOAL	JUDGMENT SCORE
Supervised operator system	**~68%**
Unattended editing in a local project	**~50%**
Safe task-to-pull-request system	**~25–38%**
Safe task-to-production system	**~12%**
Secure mobile control plane	**~10%**
Overall safe production autonomy	**~35–38%**

This does **not** mean that 62% of the source code is missing. The missing pieces are the most important safety loops: deciding what work is allowed, keeping workers separate, checking their work independently, publishing it safely, and undoing a bad release.

A simple analogy

Imagine Axon-X as a small software company.

It already has:

- an office;
- several employees;
- computers and tools;
- a security notebook recording some actions;
- a manager's control screen;
- access to the DashPro filing cabinet.

What it does **not** yet have is:

- a proper list of approved jobs;
- a manager who divides each job between employees;
- a separate desk for every employee;

- a rule preventing two employees from changing the same document;
- a quality inspector who must approve every result;
- an automatic delivery department;
- a safe test environment before customer release;
- strong identity checks at every entrance.

At present, the employees can work, but several of them may work in the same folder and leave many unfinished changes mixed together. That is useful automation, but it is not yet dependable autonomy.

What “fully autonomous” should mean

For this assessment, a fully autonomous DashPro workflow means:

1. Axon-X notices an approved task or problem.
2. It decides which specialist should handle it.
3. It gives that specialist a separate safe copy of the project.
4. The specialist makes the change.
5. Tests check whether the change really works.
6. A different AI reviewer inspects the change for mistakes.
7. Axon-X creates a clean branch and pull request.
8. It watches the online CI checks.
9. If CI fails, it attempts a limited repair.
10. It deploys to a safe staging environment.
11. It checks whether staging is healthy.
12. It either asks for production approval or automatically rolls back.
13. Every decision has a receipt explaining what happened.
14. Every agent shift (including Verifier and independent review) ends with the mandatory Critical Review Clause and `Confidence: N/10` before completion.

“Fully autonomous” should **not** mean “unlimited authority.”

Humans should continue approving:

- production secrets;
- changes to Axon-X's own security rules;
- destructive database changes;
- force-pushes;
- protected-branch merges;
- Play Store or App Store promotion;
- irreversible migrations;
- any request that expands the AI system's own authority.

The correct target is therefore **bounded autonomy**: Axon-X works alone inside carefully defined boundaries, while dangerous or irreversible actions remain protected.

What is genuinely working now

1. DashPro is connected to its real project

Axon-X is not working against a fake DashPro folder. The configuration points to the real project:

```
"workspace_dashpro": {
  "display_name": "DashPro",
  "project_root": "/home/edp/Projectx/product/dashpro"
}
```

Source: `config/workspace-project-bindings.json`

In plain language: when the DashPro workspace is selected, Axon-X knows which real folder contains the DashPro app.

2. DashPro has named AI employees

The configured DashPro company includes:

EMPLOYEE	ROLE	INTENDED RESPONSIBILITY
Dana	Lead	Priorities, decisions, and hand-offs
Cass	Watcher	Health, signals, and failed-build alerts
Priya	Frontend	Screens, user experience, Expo, and Android configuration
Marco	Backend	APIs, services, and quality failures
Soren	Integrations	GitHub Actions, runners, SDKs, and secrets wiring

These are not separate human accounts. They are different AI worker identities with different instructions.

3. Axon-X has a real scheduler

The scheduler wakes up regularly and starts workers whose schedule is `always_on` or `continuous`.

Important limits already exist:

```
DEFAULT_TICK_SECONDS = 45.0
DEFAULT_MAX_STARTS_PER_TICK = 2
DEFAULT_MAX_ACTIVE_EXECUTING = 4
```

Source: `services/control-plane/app/workspace_agents/scheduler.py`

This means:

- the scheduler checks for work approximately every 45 seconds;
- it does not start an unlimited number of workers at once;
- it currently allows at most four executing employee runs globally.

Fresh installations keep these workers off until someone enables them:

```
def default_settings() -> dict[str, Any]:
    # Safe default: continuous workers stay off until an operator enables them in UI.
    return {
        "scheduler_enabled": False,
        "max_active": 4,
        "max_starts_per_tick": 2,
        "employee_enabled": {},
    }
```

Source:
`services/control-plane/app/persistence/worker_scheduler_settings_store.py`

During this assessment, the **live scheduler was enabled**, even though its factory default is off.

4. Workers can really edit files

Scheduled workers use the same Agent execution path as the IDE and request full workspace access:

```
lane_b_result = generate_lane_b_result(
    context=context,
    user_prompt=prompt,
    run_id=run_id,
    execution_access="full",
)
```

Source:
`services/control-plane/app/workspace_agents/worker_dispatch.py`

In plain language: this is not only a chat reply. The worker can be allowed to use tools and modify the project.

5. Runs and receipts are real

Axon-X records work as a run with states such as:

```
queued
starting
executing
review_ready
awaiting_approval
completed
failed
cancelled
```

This is important because the UI does not have to guess what an agent is doing.
The control plane stores the state and history.

6. A basic mobile browser screen already exists

Axon-X already has an `OperatorMobileShell.vue` component. It shows:

- a briefing;
- fleet health;
- current signals and runs;
- KAIRO conversation controls;
- the Cloudflare tunnel URL;
- tunnel start and stop buttons.

A simplified extract:

```
<section class="operator-mobile-shell__fleet">
  <h2>Fleet</h2>
  <!-- workspace health and active runs -->
</section>

<section class="operator-mobile-shell__voice">
  <KairoGalaxyOrb />
  <KairoConversationBar />
</section>

<section class="operator-mobile-shell__tunnel">
  <!-- remote URL and tunnel controls -->
</section>
```

Source:
`apps/console-web/src/components/shell/OperatorMobileShell.vue`

This is a promising starting point, but it is still a browser page. It is not yet an installable PWA or native phone application.

What the live inspection found

The following results came from the running system and current project folders on **20 July 2026**, not only from documentation. They are **time-stamped evidence**, not permanent facts. Re-verify before using them as current state (for example, scheduler enablement and DashPro dirty-tree size can change).

FINDING	WHAT IT MEANS
Control plane and watch service were ready	The basic Axon-X stack was running (`mode: bootstrap`)
Watch reported 4 of 4 configured connectors healthy	The configured connector checks were passing
DashPro resolved to its real folder	The workspace binding is working
Scheduler was enabled	Continuous workers were allowed to run (`max_active=4` , live `max_starts_per_tick=1`)
Four of four worker slots belonged to Axon-X	DashPro had no executing employee run at that moment
DashPro workers last failed on Cursor usage limits	Latest role outcomes showed `ActionRequiredError` / out-of-usage text
Many recent DashPro employee runs were cancelled after control-plane restarts	Resume-after-restart is weak; cancelled count was high in the live store
DashPro porcelain: 59 tracked/staged + 68 untracked = 127 paths	Large mixed dirty tree in the live checkout
Tracked DashPro diff shortstat	3,939 insertions / 4,981 deletions across 59 files
Console TypeScript checking passed	`vue-tsc --noEmit` succeeded during the audit
Targeted autonomy tests	One combined run reported failures; a later isolated re-run of three modules passed (26 OK). Treat as dirty-worktree / snapshot debt until a clean baseline is recorded

The most important warning is the shared DashPro checkout.

Multiple workers are **allowed** to work in:

```
/home/edp/Projectx/product/dashpro
```

That is like allowing several employees to edit the same only copy of a document at the same time. Their changes can overlap, conflict, or become impossible to attribute to one task.

Important caveat: the 127 dirty paths were **not** proven to be caused only by continuous workers. Human edits, IDE Agent Dock turns, and cancelled worker shifts can all contribute. The hazard is the shared live checkout policy, not a fully attributed blame map.

Why the current workers are not yet a complete autonomous company

Problem 1: workers choose their own task

The worker prompt currently says:

```
"Inspect the workspace, pick the highest-value in-scope task, do it with receipts, "
"and summarize what changed. Stay inside your role boundary."
```

Source: `services/control-plane/app/workspace_agents/worker_prompt.py`

This sounds sensible, but “highest-value” is subjective.

There is no durable task record saying:

```
task_id: dashpro-142
goal: Fix checkout failure on annual plans
allowed_files:
  - app/pricing/**
  - lib/payments/**
acceptance_tests:
  - npm test -- payments
  - npm run typecheck
risk: medium
owner: backend
attempt_limit: 3
```

The YAML above is an example of what Axon-X needs. It is not a current Axon-X format.

Problem 2: the Lead does not yet manage the company

DashPro has a Lead identity, but the Lead is `on_demand`. The scheduler skips lead roles.

The Lead does not yet automatically:

- read a backlog;
- split a goal into tasks;
- decide which worker goes first;
- stop two workers from changing the same files;
- resolve dependencies;
- cancel work that is no longer needed.

The current system therefore has named employees but not yet a complete management loop.

Problem 3: ordinary workers use the live project folder

Axon-X already knows how to create disposable Git worktrees for its safe-improvement feature. That is a strong building block.

However, ordinary continuous workers do not use that isolation by default.

The safer arrangement should look like this:

```
DashPro main checkout
    ├── untouched operator copy
    ├── worktree/task-142-pricing-fix
    ├── worktree/task-143-ci-repair
    └── worktree/task-144-android-config
```

Each worker gets its own folder and branch. If a worker fails, its folder can be deleted without damaging the operator's copy.

Problem 4: verification is encouraged, not always enforced

Axon-X has many test commands and CI gates. That is good.

But a continuous worker can still finish a run after one Agent turn. The worker is instructed to test its work, but there is not yet one mandatory, machine-enforced verifier for every task.

The future rule should be:

```
if not acceptance_tests_passed:
    task.status = "failed"
    task.may_create_pull_request = False
```

This is illustrative pseudocode, not current source code.

Problem 5: no complete pull-request loop

Axon-X can commit and push when an operator explicitly asks.

It does not yet automatically:

- create one branch for one task;
- make a clean scoped commit;
- open a draft pull request;
- attach test evidence;
- read pull-request comments;
- repair review findings;
- monitor CI checks;

- update the same pull request;
- clean up a failed task branch.

This is one reason the DashPro changes have accumulated in the live checkout.

Problem 6: no staging deployment and rollback loop

Axon-X has local startup scripts, service definitions, and deployment documentation.

It does not yet have a general application pipeline that:

1. builds a fixed release artifact;
2. deploys it to staging;
3. checks staging health;
4. sends a small amount of test traffic;
5. detects a regression;
6. restores the previous version automatically.

Without that loop, it should not deploy DashPro or itself autonomously.

Problem 7: remote security is not production-ready

The FastAPI control-plane entrypoint currently adds CORS but no authentication middleware:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=_cors_origins(),
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
    allow_headers=["*"],
)
register_routes(app)
```

Source:
`services/control-plane/app/main.py`

On localhost, this can be acceptable for a trusted single operator.

It is not safe to expose powerful mutation, terminal, vault, scheduler, or tunnel actions to the internet without:

- login and session handling;
- role-based permissions;
- device registration;
- request protection;
- rate limits;
- signed approvals;
- an audit identity for every action.

Scheduled Cursor execution also currently uses broad trust flags:

```
command = [
    binary,
    "agent",
    "--print",
    "--trust",
    "--output-format",
    "stream-json",
]

if research_is_available:
    command.extend(["--force", "--approve-mcps"])
```

Source: `services/control-plane/app/cli_runtime/cursor_agent.py`

Those flags may be useful for a trusted local operator session. Scheduled workers need a narrower policy before internet-exposed autonomy is considered.

Gate 2 progress (20 July 2026, `feat/autonomous`): mutating control-plane routes now run through `MutatingAuthMiddleware` (`AXON_WATCH_AUTH_MODE=local_token` requires a bearer operator token; loopback bypass is configurable). Continuous workers dispatch Cursor with `trust_policy=worker` (keeps `--trust`, omits `--force` / `--approve-mcps`). Vault auto-unlock enable/startup is refused when the deployment is marked remotely reachable. Always-on install now forces `local_token` + `AUTH_ALLOW_LOOPBACK=0` and mints `AXON_WATCH_OPERATOR_TOKEN`; console vite `/api` proxy injects the bearer so the SPA keeps working.

Remaining Gate 2 debt: watch-service mTLS/service identity, CSRF/rate limits, and step-up Full Access keyed to a real login session (not only a shared operator token).

Gate 3 progress: continuous workers create a disposable git worktree/clone via `safe_improvement.isolated_executor` before Lane B dispatch, pass that path as `workspace_root` (never the live binding), and clean up with receipts.

Strict ordered plan

The order matters. Later stages assume the earlier safety controls already exist.

ORDER	PRIORITY	MILESTONE	PLAIN-LANGUAGE MEANING	FINISHED WHEN	ESTIMATE
0	P0	Pause and preserve	Stop adding unattended changes while the current DashPro work is sorted	Every changed file has an owner, task, and keep/discard decision	1–2 days
1	P0	Restore a trustworthy baseline	Repair the failing autonomy tests and record one known-good commit	Backend tests, frontend tests, typecheck, build, and contracts all pass together	2–4 days
2	P0	Authentication and containment	Put identity checks around powerful APIs; keep Watch internal; harden vault and Cursor trust	Anonymous users cannot mutate anything and every action has an identity	7–12 days
3	P0	Separate workspace per task	Give every worker its own branch and worktree	Two workers can work safely without sharing or changing the operator checkout	4–7 days
4	P0	Durable task ledger	Create a real list of approved tasks, owners, limits, and acceptance tests	Every run belongs to exactly one recorded task	4–7 days

ORDER	PRIORITY	MILESTONE	PLAIN-LANGUAGE MEANING	FINISHED WHEN	ESTIMATE
5	P0	Working Lead manager	Make the Lead divide goals, order dependencies, and prevent overlap	One goal becomes an ordered task plan with assigned specialists	5–8 days
6	P0	Mandatory quality gate	Tests are chosen before work starts and cannot be weakened by the worker	Failed checks always block completion and publishing	5–8 days
7	P1	Independent AI review	A different agent reviews the worker's change	Every change has a separate review result and limited repair attempts	3–5 days
8	P1	Automated pull requests	Create clean commits, branches, draft PRs, evidence, and cleanup	Every successful task ends in a reviewable PR, not a dirty folder	4–7 days
9	P1	Closed CI repair loop	Watch GitHub checks, repair failures, and rerun the exact check	A deliberately broken test is detected and repaired without a new prompt	5–8 days

ORDER	PRIORITY	MILESTONE	PLAIN-LANGUAGE MEANING	FINISHED WHEN	ESTIMATE
10	P1	Durable and fair scheduling	Resume safely after restarts and divide capacity fairly between projects	One project cannot occupy all workers; restarts do not duplicate work	5–8 days
11	P1	Staging and rollback	Test the actual running app before production	A bad staging release is detected and restored automatically	7–12 days
12	P1	DashPro autonomy trial	Run 20 real but bounded tasks and measure the results	At least 90% succeed, with no live-tree corruption or unauthorized action	2–3 weeks elapsed
13	P2	Installable mobile PWA	Turn the existing mobile page into a secure installable web app	A registered phone can inspect evidence, stop work, and approve exact actions	2–4 weeks
14	P2	Bounded production autonomy	Permit only reversible, pre-approved production actions	Rollback drills pass and all dangerous effects remain protected	2–4 weeks elapsed

Why authentication is before the mobile app

A phone control plane is not only a smaller screen. It creates a remote door into powerful systems.

Building the mobile interface first would make it easier to reach unsafe APIs. The correct order is:

```
Identity and permissions
    → safe remote connection
    → registered mobile device
    → read-only evidence
    → stop controls
    → exact approvals
    → carefully bounded mutations
```

What a future DashPro task should look like

Current simplified flow

```
Scheduler wakes worker
    → worker looks around the shared DashPro folder
    → worker chooses something it considers valuable
    → worker changes files
    → worker reports what it did
```

Weaknesses:

- the task may not be the operator's highest priority;
- another worker may edit the same files;
- tests may not be mandatory;
- the changes may remain uncommitted;
- a restart may cancel the run;
- there may be no clean pull request.

Target flow

```
Approved DashPro goal
    → Lead creates tasks and dependencies
    → task is leased to one specialist
    → isolated branch/worktree is created
    → specialist implements the task
    → mandatory checks run
    → independent reviewer inspects the diff
    → draft pull request is opened
    → GitHub CI is monitored
    → limited repair loop runs if needed
    → staging deploy is tested
    → human approves high-risk production action
    → release or automatic rollback
```

Example

An approved goal could be:

> Fix annual-plan checkout so that users cannot be charged using the wrong return URL.

Axon-X should convert that goal into something like:

```
task:
  id: dashpro-payments-018
  owner_role: backend
  risk: high
  allowed_paths:
    - lib/payments.ts
    - supabase/functions/payments-create-checkout/**
    - tests/edge/**
  acceptance:
    - payment unit tests pass
    - edge contract tests pass
    - typecheck passes
    - no secret values appear in the diff
  publication:
    create_draft_pr: true
    merge_to_main: false
    deploy_to_production: false
  limits:
    max_attempts: 3
    max_minutes: 30
```

Again, this is an example of the desired task contract, not an implemented file format.

Mobile control plane recommendation

What exists

Axon-X already has:

- a `/mobile` browser route;
- a compact interface;
- briefing information;
- fleet health;
- KAIRO conversation controls;
- tunnel visibility and controls;
- a generic mobile-push webhook adapter.

What is missing

It does not yet have:

- installable PWA packaging;
- a service worker and web manifest;
- secure device registration;
- production login;
- role-based mobile permissions;
- device revocation;
- APNs or Firebase Cloud Messaging integration;
- reliable offline and reconnect handling;
- push links that open the exact run or approval;
- a safe phone-specific approval experience.

Recommended approach

Build an authenticated **PWA first**.

A PWA is a website that can be installed on a phone and opened like an app. It allows Axon-X to reuse its existing Vue interface while the security and remote control model are proven.

Only build a separate native application later if the PWA cannot provide a required capability.

The first mobile release should support:

1. read fleet health;
2. read active-run evidence;
3. receive high-priority alerts;
4. stop a run;
5. approve or reject one exact, clearly described action;
6. revoke a lost phone.

It should not initially support unrestricted terminal access or broad production mutations.

Estimated delivery time

These are rough engineering estimates, not promises.

Assumptions:

- several areas can be developed in parallel;
- Cursor model capacity is available;
- current dirty work is triaged first;
- the project does not add major new requirements midway;
- production remains human-gated during the first trials.

TARGET	APPROXIMATE FOCUSED TIME
Restore trustworthy baseline	Less than 1 week
Safe isolated worker foundation	2–4 weeks
Dependable task-to-draft-PR autonomy	8–12 weeks
CI repair, staging, rollback, and repeated canary proof	12–16 weeks
Secure installable mobile control plane	12–20 weeks total
Optional separate native application	Additional 4–8+ weeks

The calendar time may be longer because the system needs repeated real-world trials. Reliability cannot be proven only by writing more code.

The most important immediate action

Do not add more unattended changes to the shared DashPro checkout until the current work is understood.

The immediate sequence should be:

1. pause continuous mutation;

2. preserve all current changes;
3. map files to the run or employee that changed them;
4. separate valid work into task-specific branches;
5. identify incomplete or conflicting changes;
6. restore a green verified baseline;
7. only then resume bounded workers.

This assessment did **not** pause the scheduler or change repository state. It only inspected the current system and created this document.

Terms explained

TERM	SIMPLE MEANING
Agent	An AI worker that can reason and sometimes use tools
API	A controlled doorway through which software sends requests
Authentication	Proving who is making a request
Authorization	Deciding what that person or worker is allowed to do
Branch	A separate line of changes in Git
CI	Automatic tests that run when code is proposed
Control plane	The system used to observe and control other work
Deploy	Put a version of the application where people can use it
Git worktree	A separate folder linked to the same Git project and branch history
PWA	An installable website that behaves like a phone app
Pull request	A proposed group of code changes for review
Receipt	A stored record explaining an action and its result
Rollback	Return to the previous known-good version
Scheduler	A service that decides when workers should start
Staging	A safe environment used to test a release before production
Vault	Encrypted storage for secrets and credentials

Main evidence files

CONCERN	FILE
DashPro project connection	`config/workspace-project-bindings.json`
DashPro employee roster	`config/workspace-agents.json`
Continuous worker scheduler	`services/control-plane/app/workspace_agents/scheduler.py`
Worker execution	`services/control-plane/app/workspace_agents/worker_dispatch.py`
Worker instructions	`services/control-plane/app/workspace_agents/worker_prompt.py`
Scheduler settings	`services/control-plane/app/persistence/worker_scheduler_settings_store.py`
Cursor CLI execution	`services/control-plane/app/cli_runtime/cursor_agent.py`
Agent routing	`services/control-plane/app/chat/lane_b_agent.py`
Git commit and push path	`services/control-plane/app/chat/lane_b_git_dispatch.py`
Run lifecycle	`services/control-plane/app/runs/service.py`
Safe isolated improvements	`services/control-plane/app/safe_improvement/isolated_executor.py`
Self-improvement safety rules	`docs/SELF_IMPROVEMENT_CONTRACT.md`
Mobile browser shell	`apps/console-web/src/components/shell/OperatorMobileShell.vue`
Current mobile limits	`docs/PARITY_C3_MOBILE_COMPACT_VIEWPORT.md`

CONCERN	FILE
Mobile push adapter	`services/axon-watch/app/delivery/adapters/mobile_push.py`
Dedicated-server limitations	`docs/DEDICATED_SERVER_READINESS.md`

Final conclusion

Axon-X has already crossed the line from “interface mockup” to “working supervised automation platform.”

The remaining gap is not mainly about making the AI smarter. It is about building a disciplined company around the AI:

- approved tasks;
- separate workspaces;
- clear ownership;
- independent quality checks;
- clean pull requests;
- fair scheduling;
- restart recovery;
- strong identity and permissions;
- staging and rollback;
- measured real-world trials.

Once those controls exist, DashPro can safely move from occasional AI-assisted work to continuous bounded autonomy. The same controls should then be reused by Axon-X while it builds and operates its mobile control plane.